

Proposal Review Protocol

Reviewing for _____ & _____

Work only its author has read is a guess. Review is how engineers find the flaw before it gets expensive — and it runs on the same three moves whether the artifact is a proposal, a design, or a pull request. **This is the protocol we use all semester** (design review, code review, the repo audit); today it runs on proposal drafts.

As a reviewer

1. **Read silently**, rubric in front of you, pen on the author's page. Start with **scope realism** and **risks** — where proposals fail — then problem & users.
2. **Mark three things in writing**: strongest section and why · sharpest concern, in **criteria language** · one suggestion they could act on this week. **Initial your comments**.
3. **Discuss**. A question lands better than a verdict: "*what evidence supports this?*" beats "*this seems thin.*"

As an author

- **Capture, don't defend**. You decide later what to act on — but you can't act on what you argued away.
- **Probe for the specific**. "Which paragraph made you doubt it?" turns a worry into an edit.
- **Keep both marked-up copies**. They are the evidence for **Argument & revision**, submitted with CP3.
- **Both readers come in cold** — round 2 is an *independent* read, and adds one question: "**Would this pitch recruit you?**"

1 · The register — say it so it can be used

USABLE — SPECIFIC, CRITERIA-ANCHORED
NOT USABLE — TASTE, OR A SHRUG

"Scope realism — I don't believe the calendar sync fits five weeks of real building."

"This seems like a lot."

"Problem & users — 'students' isn't bounded. Which ones, how many, how did you reach them?"

"The problem section is vague."

"Risks — these could be in anyone's proposal. Which one is specific to *this* stack?"

"Good risks section."

2 · What you are reading for — the CP3 rubric

The two criteria marked ★ are where proposals fail — start your read there, then work the rest. Name the criterion when you raise a concern; that is what makes feedback actionable instead of personal.

CRITERION	WT	STRONG LOOKS LIKE
Problem & user clarity	15	A compelling, evidence-backed problem and a bounded userbase — not "students" or "everyone."
Solution & core features	15	A coherent solution, a sharp ranked feature set, and explicit non-goals .
Feasibility & approach	20	Stack and dependencies actually researched; the approach is justified, not asserted.
★ Scope realism (MVP)	15	A vertical slice sized against the <i>real</i> build window — roughly five weeks — with the arithmetic shown .
Richness & extensibility	15	Exercises several engineering dimensions; a credible path for others to build on it.
★ Risks & mitigation	10	Real, project-specific risks with mitigations that carry a date or a trigger.
Argument & revision	10	Persuasive writing with visible response to peer feedback — measured against today's draft.

Argument & revision requires a draft on file. Today's LMS upload is what your revision is measured against; with no draft on file the criterion caps at Developing.

3 · If the finding is "too big" — the narrowing moves

- **Cut the platform, keep the slice** — one user type, one workflow, end to end.
- **Demote features to non-goals** — naming what you are *not* building strengthens the proposal.
- **Shrink the user** — one club, one ward, one lab, one theater.
- **Defer the hard integration** — v1 proves the core; the sync is the roadmap's job.

Then check the arithmetic out loud. The ~10 hrs/week is *everything* — class, reading, discovery, writing, reviews. Subtract those first, then multiply what's left by **roughly five weeks of real building**. Most overscoped proposals die on this line.

4 · Your revision list — three edits, ranked

Start with anything **both** readers flagged — that one is the proposal, not the reader.

1

2

3

The one narrowing move my proposal most needs: _____

Tonight: scan or photograph **both** copies — a scan is cleaner, a photo is fine if legible. Submit both images with CP3.

CS 301R · Studio I